

LAPORAN PRAKTIKUM
PRAKTIKUM PENAMBANGAN DATA
PERTEMUAN 5
FORECASTING



Disusun oleh:

Nama : Hafidz Rizqullah Prasetya
NIM : 24/535493/SV/24243
Kelas : PL5A1
Dosen Pengampu : Dr. Imam Fahrurrozi, S.T., M.Cs.

PROGRAM STUDI D-IV
TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026

Daftar Isi

Daftar Isi	2
Tujuan Praktikum	3
Dasar Teori	4
2.1 Supervised Learning	4
2.2 Forecasting	4
2.3 Sliding Window	5
2.4 Feature Engineering: Fitur Statistik	5
2.5 Algoritma Machine Learning	5
2.5.1 Multilayer Perceptron (Neural Network)	5
2.5.2 K-Nearest Neighbors (KNN)	5
2.5.3 Decision Tree	6
2.5.4 Support Vector Regression (SVR)	6
2.5.5 Random Forest	6
2.6 Metrik Evaluasi	6
2.6.1 Root Mean Squared Error (RMSE)	6
2.6.2 Pearson Correlation Coefficient	6
2.7 Validasi dan Pencegahan Data Leakage	7
Hasil dan Pembahasan	8
3.1 Percobaan Latihan Praktikum	8
3.1.1 Alat dan Bahan	8
3.1.2 Persiapan dan Impor Pustaka	8
3.1.3 Fungsi Sliding Window	9
3.1.4 Fungsi Fitur Statistik	9
3.1.5 Memuat Dataset	10
3.1.6 Membentuk Time Series dan Preprocessing	11
3.1.7 Visualisasi Deret Waktu	12
3.1.8 Pelatihan Model dan Prediksi	13
3.1.9 Visualisasi Hasil Prediksi	13
3.1.10 Evaluasi Model	14
3.1.11 Perbandingan dengan Baseline Naif	15

3.2	Tugas dan Analisis: Forecasting Harga Saham Tesla	17
3.2.1	Deskripsi Dataset dan Atribut	17
3.2.2	Skenario A: Prediksi dari Close 7 Hari Sebelumnya	18
3.2.3	Skenario B: Prediksi dari Close dan Open 7 Hari Sebelumnya	18
3.2.4	Analisis: Mengapa MLP Unggul di Sini	19
3.2.5	Perbandingan Skenario A dan B	21
Kesimpulan		22
Daftar Pustaka		24

Tujuan Praktikum

Sesuai dengan Modul Pertemuan 5, tujuan pembelajaran pada praktikum ini adalah:

1. Mahasiswa dapat menerapkan metode *forecasting* berbasis *machine learning*, meliputi Neural Network, K-Nearest Neighbors (KNN), Decision Tree, Support Vector Regression (SVR), dan Random Forest, untuk memprediksi nilai periode berikutnya berdasarkan data historis.
2. Mahasiswa dapat melakukan analisis dan perbandingan performa model menggunakan metrik evaluasi *Root Mean Squared Error* (RMSE) dan *Pearson Correlation Coefficient*.
3. Mahasiswa dapat menentukan model *forecasting* yang paling akurat berdasarkan hasil evaluasi menggunakan RMSE dan *Pearson Correlation Coefficient*.

Dasar Teori

2.1 Supervised Learning

Supervised learning adalah teknik *machine learning* yang melatih model memakai data berlabel. Sistem diberi pasangan masukan dan keluaran yang diinginkan, lalu mempelajari pola dari data tersebut untuk memprediksi nilai pada data baru. Algoritma yang termasuk di dalamnya antara lain Decision Tree, K-Nearest Neighbor, Naive Bayes, regresi, dan Support Vector Machine [1, 2]. Karena pola yang dipelajari berasal dari contoh yang sudah diketahui jawabannya, kualitas prediksi bergantung pada seberapa representatif data latih yang diberikan.

2.2 Forecasting

Forecasting adalah proses memperkirakan keadaan di masa depan dengan menguji pola pada data masa lalu [3, 4]. Hasil ramalan membantu pengambilan keputusan, misalnya menentukan jumlah stok barang atau perencanaan kapasitas. Ramalan bisa bersifat kualitatif, seperti perkiraan bahwa besok cuaca cerah, atau kuantitatif berupa angka. Praktikum ini mengerjakan jenis yang kedua karena targetnya berupa nilai kontinu.

Peramalan deret waktu berbeda dari regresi biasa karena urutan waktu data tidak boleh diacak. Nilai hari ini bergantung pada nilai beberapa hari sebelumnya, sehingga susunan data saat pelatihan harus tetap kronologis. Modul membedakan dua jenis peramalan berdasarkan jarak prediksinya:

- **One-step forecast:** model memprediksi satu nilai untuk satu langkah waktu berikutnya.
- **Multi-step forecast:** model memprediksi beberapa langkah waktu sekaligus, misalnya dua hari ke depan seperti pada tugas praktikum ini.

2.3 Sliding Window

Untuk mengubah deret waktu menjadi format *supervised learning*, dipakai teknik *sliding window*. Jendela berukuran tetap digeser satu langkah demi satu langkah sepanjang deret, sehingga setiap posisi menghasilkan satu pasangan masukan dan target [28, 4]. Misalnya dengan ukuran jendela 7, nilai tujuh hari pertama menjadi masukan dan nilai hari ke-8 menjadi target, lalu jendela digeser satu hari untuk membentuk pasangan berikutnya.

Pendekatan ini dipakai karena algoritma regresi pada umumnya menerima tabel fitur, bukan urutan. Setelah diubah, pola urutan waktu bisa dipelajari oleh model seperti Linear Regression, Decision Tree, maupun Random Forest.

2.4 Feature Engineering: Fitur Statistik

Fitur statistik ditambahkan pada tiap jendela agar model mendapat gambaran distribusi data di dalamnya. Modul menyebut delapan fitur: nilai minimum, maksimum, selisih maksimum dan minimum, rata-rata, standar deviasi, median, kurtosis, dan skewness. Min, max, dan selisih menggambarkan rentang nilai; mean, median, dan standar deviasi menggambarkan pusat dan sebaran; sedangkan kurtosis dan skewness menggambarkan bentuk distribusi [29, 18, 19]. Dengan tujuh nilai mentah ditambah delapan fitur statistik, satu sampel menjadi 15 fitur.

2.5 Algoritma Machine Learning

Modul memakai lima algoritma regresi. Linear Regression tidak dipakai di sini karena modul menyoroti lima model berikut.

2.5.1 Multilayer Perceptron (Neural Network)

MLPRegressor adalah jaringan saraf tiruan berlapis yang tersusun dari unit masukan, satu atau lebih lapisan tersembunyi, dan unit keluaran. Setiap koneksi punya bobot yang diperbarui lewat *backpropagation* untuk menekan kesalahan prediksi [7, 23]. Jaringan ini cukup lentur menangkap hubungan non-linear, tetapi hasilnya bisa berbeda tiap kali dijalankan bila `random_state` tidak dikunci. Karena itu modul memakai `random_state=42` agar hasilnya dapat direproduksi.

2.5.2 K-Nearest Neighbors (KNN)

KNeighborsRegressor memprediksi nilai sebuah titik dengan merata-ratakan target dari k tetangga terdekatnya [8, 24]. Model ini tidak membangun persamaan saat pelatihan, seluruh data latih disimpan dan dibandingkan saat prediksi. Nilai k bawaan adalah 5. Kelebihan

KNN adalah kesederhanaannya, sedangkan kelemahannya ia sensitif terhadap skala fitur dan menjadi lambat ketika data latih membesar.

2.5.3 Decision Tree

DecisionTreeRegressor membagi data secara rekursif berdasarkan titik pemisah yang paling menurunkan kesalahan, biasanya diukur dengan *Mean Squared Error*. Setiap daun berisi nilai prediksi berupa rata-rata target di dalamnya [9, 25]. Pohon tunggal mudah dibaca, tetapi rawan *overfitting* karena bisa tumbuh terlalu dalam mengikuti data latih.

2.5.4 Support Vector Regression (SVR)

SVR mencari fungsi yang menyimpang paling jauh sebesar ε dari target, sambil meminimalkan kompleksitas model [10, 26]. Titik data yang berada di dalam batas ε diabaikan dari fungsi biaya, sehingga SVR toleran terhadap derau kecil. Karena bekerja dengan jarak, fitur sebaiknya dinormalisasi lebih dulu agar tiap variabel memberi kontribusi seimbang.

2.5.5 Random Forest

RandomForestRegressor membangun banyak pohon keputusan dari sampel *bootstrap* dan subset fitur acak, lalu merata-ratakan prediksinya [11, 27]. Cara ini meredam kelemahan pohon tunggal, sehingga hasilnya lebih stabil dan tidak mudah *overfitting*. Harganya adalah waktu latih dan kebutuhan memori yang lebih besar.

2.6 Metrik Evaluasi

2.6.1 Root Mean Squared Error (RMSE)

RMSE mengukur rata-rata jarak antara nilai prediksi dan nilai aktual, dengan penalti lebih besar pada kesalahan yang besar. RMSE adalah akar dari *Mean Squared Error*, sehingga satuannya sama dengan satuan target [12, 5].

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_{pred,i} - y_{true,i})^2} \quad (.1)$$

Semakin kecil RMSE, semakin akurat model. Karena kesalahan dikuadratkan, RMSE peka terhadap *outlier*: satu prediksi yang jauh meleset akan menaikkan nilai RMSE cukup besar.

2.6.2 Pearson Correlation Coefficient

Koefisien korelasi Pearson (r) mengukur kekuatan dan arah hubungan linear antara nilai aktual dan nilai prediksi. Nilainya berkisar dari -1 sampai 1 . Nilai mendekati 1 berarti

prediksi mengikuti pola aktual dengan arah yang sama, mendekati -1 berarti hubungannya berlawanan, dan mendekati 0 berarti tidak ada hubungan linear [6, 17].

$$r = \frac{\sum (y_{true,i} - \bar{y}_{true})(y_{pred,i} - \bar{y}_{pred})}{\sqrt{\sum (y_{true,i} - \bar{y}_{true})^2 \sum (y_{pred,i} - \bar{y}_{pred})^2}} \quad (.2)$$

RMSE dan Pearson saling melengkapi. RMSE menunjukkan besar kesalahan, sedangkan Pearson menunjukkan seberapa baik bentuk pola prediksi menyerupai pola aktual.

2.7 Validasi dan Pencegahan Data Leakage

Data dibagi menjadi bagian latih dan bagian uji. Pada deret waktu, pembagian tidak boleh diacak karena akan membuat model "melihat" masa depan. Modul memakai `shuffle=False` pada `train_test_split` agar 80% data awal menjadi latih dan 20% data akhir menjadi uji [14]. Normalisasi juga di-*fit* hanya pada data latih, lalu diterapkan ke data uji, supaya informasi dari data uji tidak bocor ke proses pelatihan [13].

Hasil dan Pembahasan

3.1 Percobaan Latihan Praktikum

Bagian ini mendokumentasikan percobaan *forecasting* sesuai langkah pada Modul Pertemuan 5. Dataset yang dipakai adalah *Bike Sharing Dataset* [30], yang berisi catatan peminjaman sepeda harian beserta kondisi cuaca. Saya memuat data dari mirror GitHub [32] agar bisa langsung dibaca dengan `pd.read_csv()` tanpa unduhan manual. Notebook yang saya gunakan tersedia pada repositori ini sebagai `praktikum-5-forecasting.ipynb`.

3.1.1 Alat dan Bahan

Sesuai Modul 5.3, alat dan bahan yang saya gunakan adalah:

- **Perangkat keras:** laptop dengan RAM minimal 4 GB dan koneksi internet stabil.
- **Perangkat lunak:** Python 3 [22] dan lingkungan notebook (Google Colab atau Jupyter). Pustaka utama meliputi `pandas` [15], `NumPy` [16], `Matplotlib` [20], `Seaborn` [21], `Scikit-Learn` [12, 13], dan `SciPy` untuk perhitungan korelasi [17].

3.1.2 Persiapan dan Impor Pustaka

Saya mengimpor pustaka untuk manipulasi data, perhitungan numerik, visualisasi, pemodelan lima algoritma, serta metrik evaluasi. Blok impor juga memuat `pearsonr`, `kurtosis`, dan `skew` dari `SciPy` untuk evaluasi dan fitur statistik.

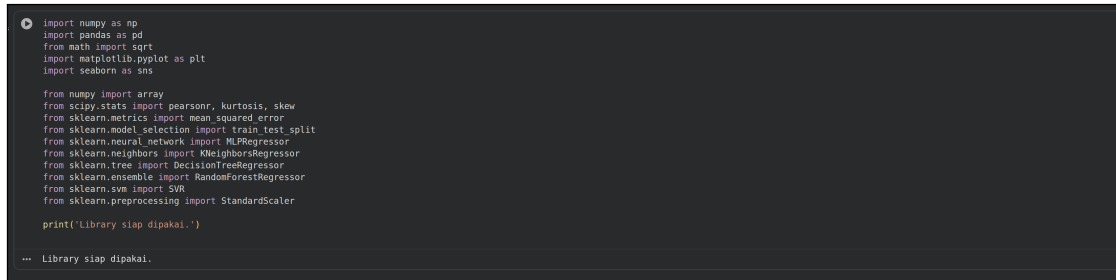
```
1 import numpy as np
2 import pandas as pd
3 from math import sqrt
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6
7 from numpy import array
8 from scipy.stats import pearsonr, kurtosis, skew
9 from sklearn.metrics import mean_squared_error
10 from sklearn.model_selection import train_test_split
11 from sklearn.neural_network import MLPRegressor
```

```

12 from sklearn.neighbors import KNeighborsRegressor
13 from sklearn.tree import DecisionTreeRegressor
14 from sklearn.ensemble import RandomForestRegressor
15 from sklearn.svm import SVR
16 from sklearn.preprocessing import StandardScaler

```

Listing .1: Impor pustaka praktikum forecasting.



```

import numpy as np
import pandas as pd
from math import sqrt
import matplotlib.pyplot as plt
import seaborn as sns

from numpy import array
from scipy.stats import pearsonr, kurtosis, skew
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.svm import SVR
from sklearn.preprocessing import StandardScaler

print('Library siap dipakai.')

```

Gambar 1: Blok impor pustaka yang dipakai sepanjang praktikum.

3.1.3 Fungsi Sliding Window

Fungsi `split_sequences` mengubah deret waktu menjadi pasangan masukan dan target. Untuk setiap posisi, ia mengambil `n_steps_in` baris sebagai masukan dan satu nilai pada `n_steps_out` langkah setelahnya sebagai target. Perulangan berhenti ketika jendela sudah melewati akhir data.

```

1 def split_sequences(sequences, n_steps_in, n_steps_out):
2     X, y = list(), list()
3     for i in range(len(sequences)):
4         end_ix = i + n_steps_in
5         out_end_ix = end_ix + n_steps_out
6         if out_end_ix > len(sequences):
7             break
8         seq_x, seq_y = sequences[i:end_ix, :-1], sequences[out_end_ix - 1, -1]
9         X.append(seq_x)
10        y.append(seq_y)
11    return array(X), array(y)

```

Listing .2: Fungsi sliding window.

3.1.4 Fungsi Fitur Statistik

Fungsi `stats_features` menghitung delapan nilai statistik pada setiap jendela, lalu menyambungkannya ke data mentah jendela tersebut. Hasilnya, satu sampel yang semula berisi tujuh nilai berubah menjadi 15 fitur.

```

1 def stats_features(input_data):
2     inp = list()

```

```

3     for i in range(len(input_data)):
4         inp2 = input_data[i]
5         min_val = float(np.min(inp2))
6         max_val = float(np.max(inp2))
7         diff = max_val - min_val
8         std_val = float(np.std(inp2))
9         mean_val = float(np.mean(inp2))
10        median_val = float(np.median(inp2))
11        kurt = float(kurtosis(inp2))
12        sk = float(skew(inp2))
13        for v in (min_val, max_val, diff, std_val, mean_val, median_val, kurt, sk):
14            inp2 = np.append(inp2, v)
15        inp = np.append(inp, inp2)
16    return inp.reshape(len(input_data), -1)

```

Listing .3: Fungsi fitur statistik.

3.1.5 Memuat Dataset

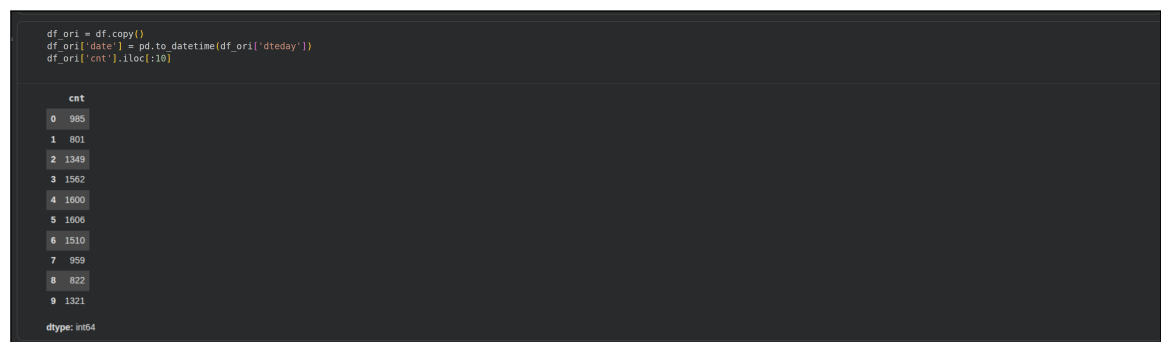
Dataset bike sharing berisi 731 baris dan 16 kolom, mencakup tanggal, kondisi cuaca, suhu, kelembapan, dan jumlah peminjaman pada kolom cnt.

```

1 df = pd.read_csv('https://raw.githubusercontent.com/ganjar87/data_science_practice/
↪ main/bikesharing_day.csv')
2 print('shape:', df.shape)
3 df.head()

```

Listing .4: Memuat dataset bike sharing.



```

df_ori = df.copy()
df_ori['date'] = pd.to_datetime(df_ori['dteday'])
df_ori['cnt'].iloc[:10]

```

	cnt
0	985
1	801
2	1349
3	1562
4	1600
5	1606
6	1510
7	959
8	822
9	1321

dtype: int64

Gambar 2: Lima baris pertama dataset bike sharing: kolom instant, dteday, season, hingga cnt.

Kolom tanggal dteday saya ubah ke format datetime supaya bisa dipakai untuk sumbu waktu pada grafik.

```

1 df_ori = df.copy()
2 df_ori['date'] = pd.to_datetime(df_ori['dteday'])
3 df_ori['cnt'].iloc[:10]

```

Listing .5: Konversi kolom tanggal dan cek sepuluh nilai target.

```
df_ori = df.copy()
df_ori['cnt'] = df_ori['cnt'].astype(float)
df_ori['cnt'] = df_ori['cnt'].astype(int)

cnt
0    985
1    801
2   1349
3   1562
4   1600
5   1606
6   1510
7    959
8    822
9   1321
dtype: int64
```

Gambar 3: Sepuluh nilai pertama kolom target cnt: 985, 801, 1349, 1562, 1600, 1606, 1510, 959, 822, dan 1321.

3.1.6 Membentuk Time Series dan Preprocessing

Masukan dibentuk dari nilai cnt tujuh hari sebelumnya untuk memprediksi hari ke-8. Saya menduplikasi kolom cnt menjadi dua kolom karena fungsi `split_sequences` memisahkan kolom terakhir sebagai target, sedangkan sisanya menjadi fitur. Data hasil jendela lalu dibagi 80:20 dengan `shuffle=False`, kemudian fitur statistik ditambahkan pada data latih dan data uji.

```
1 df_X = df_ori[['cnt', 'cnt']]
2 in_seq = df_X.astype(float).values
3
4 n_steps_in, n_steps_out = 7, 1
5 X, y = split_sequences(in_seq, n_steps_in, n_steps_out)
6
7 n_input = X.shape[1] * X.shape[2]
8 X = X.reshape((X.shape[0], n_input))
9
10 X_train, X_test, y_train, y_test = train_test_split(
11     X, y, test_size=0.2, random_state=42, shuffle=False)
12
13 X_train = stats_features(X_train)
14 X_test = stats_features(X_test)
```

Listing .6: Sliding window, split, dan penambahan fitur statistik.

```
df_X = df_ori[['cnt', 'cnt']]
in_seq = df_X.astype(float).values

n_steps_in, n_steps_out = 7, 1
X, y = split_sequences(in_seq, n_steps_in, n_steps_out)
print('X shape:', X.shape)
print('y shape:', y.shape)

n_input = X.shape[1] * X.shape[2]
X = X.reshape((X.shape[0], n_input))

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, shuffle=False)

X_train = stats_features(X_train)
X_test = stats_features(X_test)
print('X_train shape:', X_train.shape, 'X_test shape:', X_test.shape)

X_train: (724, 7, 1)
X_test: (145, 7, 1)
y_train: (724, 1)
y_test: (145, 1)
```

Gambar 4: Hasil pembentukan jendela: 724 sampel dengan tujuh nilai masukan. Sampel pertama berisi tujuh hari awal, targetnya 959,0 pada hari ke-8.

```
df = df_ori[['cnt', 'date']]
x_train = df[['cnt', 'date']]

# Split data into training and testing sets
x_train, x_test, y_train, y_test = train_test_split(
    x_train, x_test, y_train, y_test, random_state=42, shuffle=False)

# Print the shapes of the training and testing sets
print('x_train shape:', x_train.shape)
print('x_test shape:', x_test.shape)
print('y_train shape:', y_train.shape)
print('y_test shape:', y_test.shape)

# Print the first few rows of the training and testing sets
print('x_train head:', x_train.head())
print('x_test head:', x_test.head())
print('y_train head:', y_train.head())
print('y_test head:', y_test.head())

# Print the first few rows of the training and testing sets
print('x_train tail:', x_train.tail())
print('x_test tail:', x_test.tail())
print('y_train tail:', y_train.tail())
print('y_test tail:', y_test.tail())
```

Gambar 5: Bentuk data setelah fitur statistik ditambahkan: 579 sampel latih dan 145 sampel uji, masing-masing dengan 15 fitur.

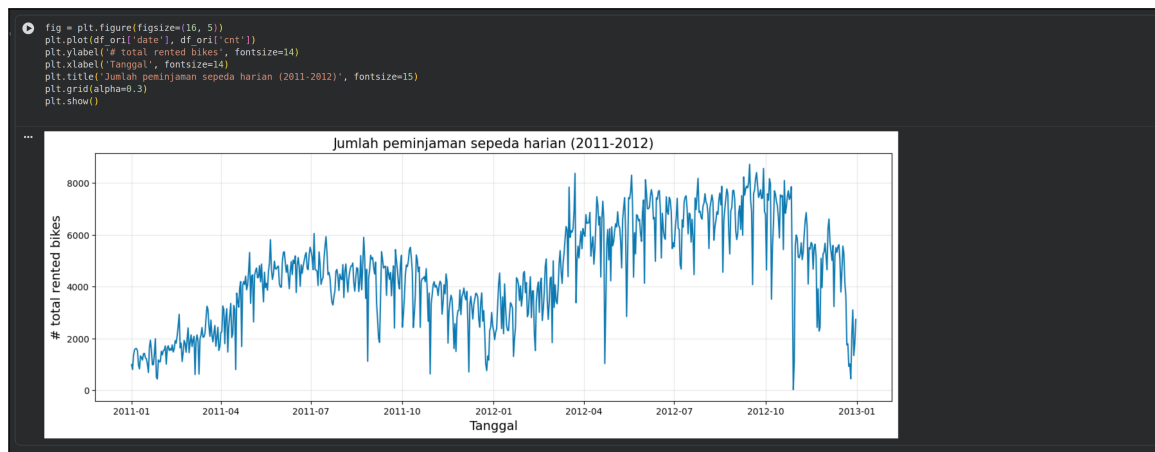
Jumlah 724 sampel berasal dari 731 baris dikurangi 7 hari yang tidak punya riwayat sebelumnya. Pembagian 80:20 menghasilkan 579 sampel latih dan 145 sampel uji, dan jumlah kolom bertambah dari 7 menjadi 15 setelah delapan fitur statistik disisipkan.

3.1.7 Visualisasi Deret Waktu

Sebelum melatih model, saya melihat pola deret waktu jumlah peminjaman sepeda untuk memahami sifat datanya.

```
1 fig = plt.figure(figsize=(16, 5))
2 plt.plot(df_ori['date'], df_ori['cnt'])
3 plt.ylabel('# total rented bikes', fontsize=14)
4 plt.xlabel('Tanggal', fontsize=14)
5 plt.show()
```

Listing .7: Visualisasi deret waktu jumlah peminjaman.



Gambar 6: Deret waktu jumlah peminjaman sepeda harian sepanjang 2011 sampai 2012. Terlihat pola musiman naik-turun dan tren yang menguat pada tahun kedua.

Grafik menunjukkan dua hal. Pertama, ada pola berulang yang menandakan pengaruh musim dan hari kerja. Kedua, jumlah peminjaman pada 2012 lebih tinggi daripada 2011, jadi datanya punya tren naik. Kedua sifat ini masuk akal untuk data peminjaman sepeda dan menjelaskan mengapa model berbasis pola historis bisa bekerja cukup baik.

3.1.8 Pelatihan Model dan Prediksi

Saya melatih lima model dengan fungsi terpisah. Tiap fungsi melatih model pada data latih, memprediksi data uji, membulatkan hasil prediksi, lalu menghitung RMSE dan Pearson.

```
1 def mlp(X_train, X_test, y_train, y_test):
2     model = MLPRegressor(random_state=42)
3     model.fit(X_train, y_train)
4     y_pred = np.round(model.predict(X_test), 0)
5     rmse = sqrt(mean_squared_error(y_test, y_pred))
6     corr, _ = pearsonr(y_test, y_pred)
7     return rmse, corr, y_pred
```

Listing .8: Fungsi model MLP (contoh, model lain mengikuti pola yang sama).

```
1 rmse_mlp, corr_mlp, y_pred_mlp = mlp(X_train, X_test, y_train, y_test)
2 rmse_knn, corr_knn, y_pred_knn = knn(X_train, X_test, y_train, y_test)
3 rmse_dt, corr_dt, y_pred_dt = dt(X_train, X_test, y_train, y_test)
4 rmse_svm, corr_svm, y_pred_svm = svm(X_train, X_test, y_train, y_test)
5 rmse_rf, corr_rf, y_pred_rf = rf(X_train, X_test, y_train, y_test)
```

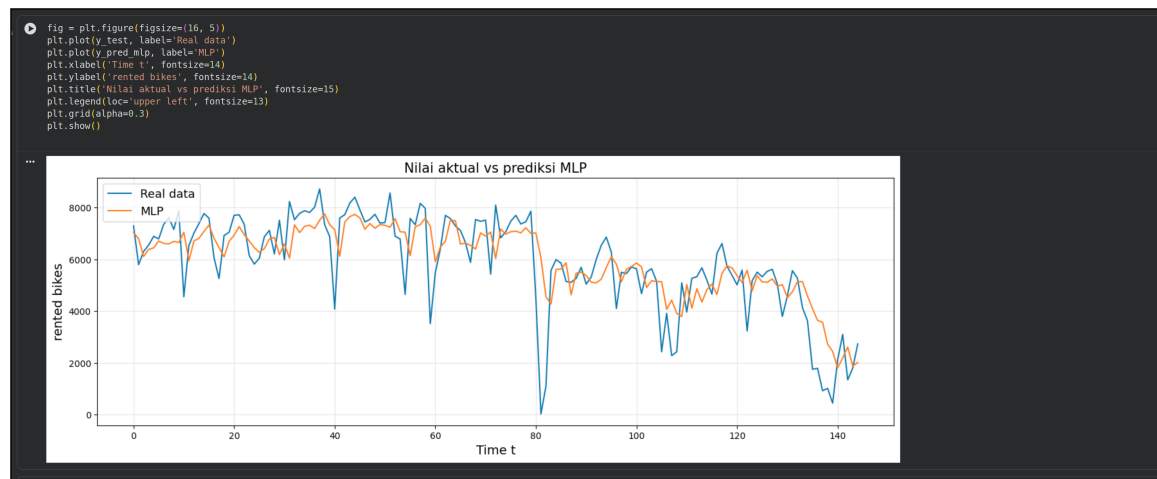
Listing .9: Memanggil kelima model.

3.1.9 Visualisasi Hasil Prediksi

Gambar berikut membandingkan nilai aktual dengan prediksi MLP pada seluruh data uji, lalu membandingkan empat model pada 100 sampel pertama agar perbedaannya lebih mudah dilihat.

```
1 fig = plt.figure(figsize=(16, 5))
2 plt.plot(y_test, label='Real data')
3 plt.plot(y_pred_mlp, label='MLP')
4 plt.xlabel('Time t', fontsize=14)
5 plt.ylabel('rented bikes', fontsize=14)
6 plt.legend(loc='upper left', fontsize=13)
7 plt.show()
```

Listing .10: Visualisasi aktual vs prediksi.



Gambar 7: Nilai aktual (biru) dibandingkan prediksi MLP (oranye) pada data uji. Kedua garis mengikuti pola yang mirip, meski prediksi tampak lebih halus dan melewati sebagian puncak.



Gambar 8: Perbandingan empat model pada 100 sampel pertama. MLP paling rapat mengikuti data aktual, sedangkan Decision Tree terlihat paling jauh dari pola.

Prediksi MLP mengikuti bentuk umum data aktual, tetapi garisnya lebih mulus. Puncak peminjaman yang tinggi cenderung diprediksi lebih rendah, karena model cenderung menahan prediksi mendekati rata-rata pola yang pernah dilihatnya. Pada perbandingan empat model, Decision Tree terlihat paling jarang menyesuaikan diri dengan pola, sementara MLP dan Random Forest lebih rapat.

3.1.10 Evaluasi Model

Evaluasi memakai RMSE dan Pearson untuk kelima model.

```

1 for name, rmse, corr in [
2     ('MLP', rmse_mlp, corr_mlp),
3     ('KNN', rmse_knn, corr_knn),

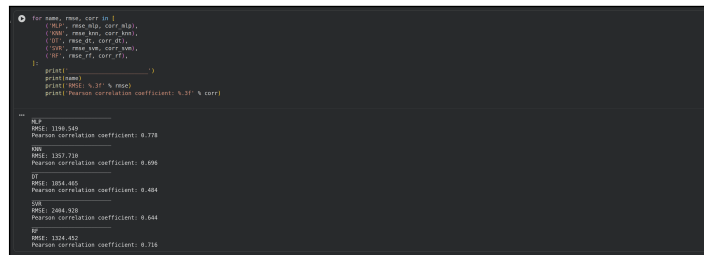
```

```

4      ('DT', rmse_dt, corr_dt),
5      ('SVR', rmse_svm, corr_svm),
6      ('RF', rmse_rf, corr_rf),
7  ]:
8      print('_____')
9      print(name)
10     print('RMSE: %.3f' % rmse)
11     print('Pearson correlation coefficient: %.3f' % corr)

```

Listing .11: Mencetak RMSE dan Pearson tiap model.



```

for name, rmse, corr in zip(
    ('MLP', rmse_mlp, corr_mlp),
    ('RF', rmse_rf, corr_rf),
    ('KNN', rmse_knn, corr_knn),
    ('DT', rmse_dt, corr_dt),
    ('SVR', rmse_svm, corr_svm)):
    print('_____')
    print(name)
    print('RMSE: %.3f' % rmse)
    print('Pearson correlation coefficient: %.3f' % corr)

```

MLP
RMSE: 1190.549
Pearson correlation coefficient: 0.778

Random Forest
RMSE: 1324.452
Pearson correlation coefficient: 0.696

KNN
RMSE: 1357.710
Pearson correlation coefficient: 0.484

Decision Tree
RMSE: 1854.465
Pearson correlation coefficient: 0.644

SVR
RMSE: 2404.928
Pearson correlation coefficient: 0.716

Gambar 9: Hasil evaluasi kelima model pada dataset bike sharing.

Model	RMSE	Pearson
MLP (Neural Network)	1190,549	0,778
Random Forest	1324,452	0,716
KNN	1357,710	0,696
Decision Tree	1854,465	0,484
SVR	2404,928	0,644

Table .1: Perbandingan performa lima model pada dataset bike sharing (split 80:20, shuffle=False).

MLP menempati urutan pertama pada kedua metrik: RMSE terendah 1190,549 dan Pearson tertinggi 0,778. Random Forest menyusul dengan RMSE 1324,452, sedikit lebih baik daripada KNN. Decision Tree jadi model terlemah dari sisi Pearson (0,484) karena pohon tunggal mudah menyesuaikan diri berlebihan pada data latih, sehingga prediksinya kurang mulus ketika menghadapi data uji. SVR memberi RMSE tertinggi 2404,928; nilai bawaannya bekerja pada skala fitur asli, sedangkan fitur di sini tidak dinormalisasi, sehingga hasilnya kurang optimal. Temuan ini sesuai dengan catatan pada bagian dasar teori bahwa SVR sebaiknya dipakai bersama normalisasi.

3.1.11 Perbandingan dengan Baseline Naif

Untuk menilai apakah model benar-benar belajar, saya membandingkan hasilnya dengan *baseline* sederhana yang memprediksi nilai besok sama dengan nilai hari terakhir pada jendela. Baseline ini hanya mengulang angka terakhir tanpa belajar pola.

```

1 last_value = X[:, -1]
2 rmse_naive = sqrt(mean_squared_error(y_test, last_value[-len(y_test):]))
3 print('naive RMSE: %.3f' % rmse_naive)

```

Listing .12: Menghitung RMSE baseline naif.

```

last_value = X[:, -1]
rmse_naive = sqrt(mean_squared_error(y_test, last_value[-len(y_test):]))
corr_naive = pearsonr(y_test, last_value[-len(y_test):])
print('naive RMSE: %.3f | Pearson: %.3f' % (rmse_naive, corr_naive))

naive RMSE: 1290.766 | Pearson: 0.763

```

Gambar 10: Output perhitungan RMSE baseline naif pada notebook.

Baseline naif menghasilkan RMSE 1290,766. MLP (1190,549) dan Random Forest (1324,452) berada di sekitar angka itu. Artinya untuk data peminjaman sepeda yang berfluktuasi harian, sebagian besar nilai prediksi yang wajar bisa didapat hanya dengan mengulang nilai terakhir. Keunggulan MLP di sini nyata, tetapi tidak besar, dan patut dibaca dengan hati-hati.

3.2 Tugas dan Analisis: Forecasting Harga Saham Tesla

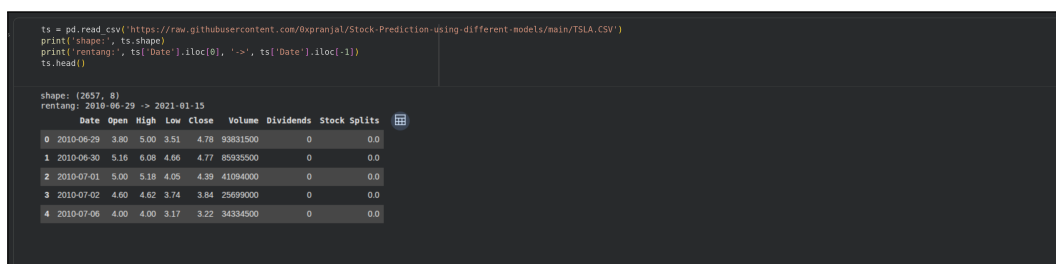
Sesuai Modul 5.5, saya mengerjakan tugas dengan dataset *Tesla Stock Data from 2010 to 2020* [31]. Dataset ini memuat harga saham Tesla harian. Karena Kaggle memerlukan unduhan manual, saya memakai mirror GitHub [33] yang memuat isi yang sama.

3.2.1 Deskripsi Dataset dan Atribut

Dataset berisi 2.657 baris dengan delapan kolom, mencakup rentang 29 Juni 2010 sampai 15 Januari 2021.

```
1 ts = pd.read_csv('https://raw.githubusercontent.com/Oxpranjal/Stock-Prediction-using-  
↪ -different-models/main/TSLA.CSV')  
2 print('shape:', ts.shape)  
3 print('rentang:', ts['Date'].iloc[0], '->', ts['Date'].iloc[-1])  
4 ts.head()
```

Listing .13: Memuat dan memeriksa dataset Tesla.



```
ts = pd.read_csv('https://raw.githubusercontent.com/Oxpranjal/Stock-Prediction-using-different-models/main/TSLA.CSV')  
print('shape:', ts.shape)  
print('rentang:', ts['Date'].iloc[0], '->', ts['Date'].iloc[-1])  
ts.head()
```

shape: (2657, 8)
rentang: 2010-06-29 -> 2021-01-15

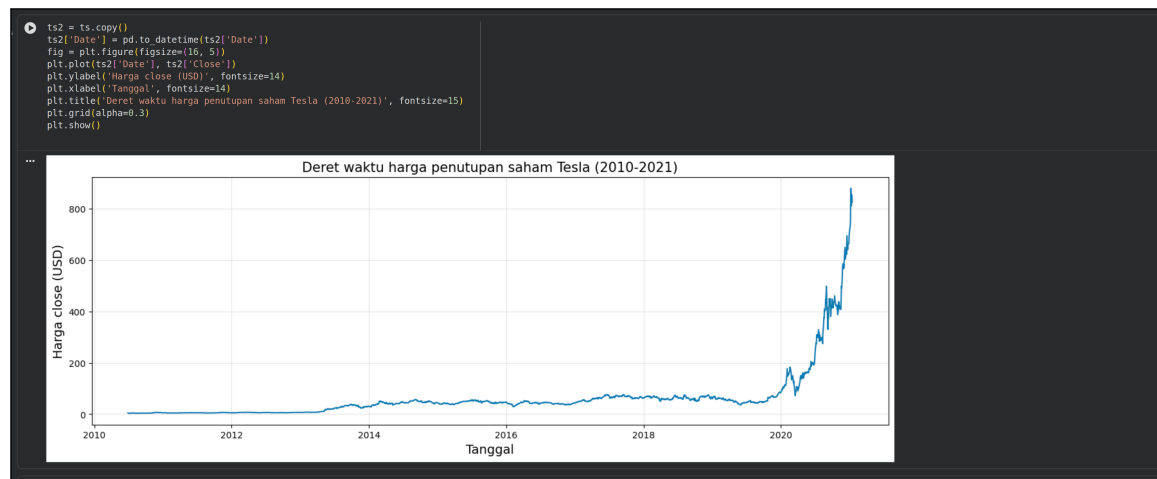
	Date	Open	High	Low	Close	Volume	Dividends	Stock Splits
0	2010-06-29	3.80	5.00	3.51	4.78	93831500	0	0.0
1	2010-06-30	5.16	6.08	4.66	4.77	85335500	0	0.0
2	2010-07-01	5.00	5.18	4.05	4.39	41094000	0	0.0
3	2010-07-02	4.60	4.62	3.74	3.84	25699000	0	0.0
4	2010-07-06	4.00	4.00	3.17	3.22	34334500	0	0.0

Gambar 11: Lima baris pertama dataset Tesla: kolom Date, Open, High, Low, Close, Volume, Dividends, dan Stock Splits.

Atribut	Penjelasan
Date	Tanggal perdagangan.
Open	Harga pembukaan pada hari tersebut.
High	Harga tertinggi hari itu.
Low	Harga terendah hari itu.
Close	Harga penutupan, menjadi target prediksi.
Volume	Jumlah saham yang diperdagangkan.
Dividends	Pembayaran dividen, seluruhnya nol pada dataset ini.
Stock Splits	Rasio pemecahan saham, seluruhnya nol pada dataset ini.

Table .2: Deskripsi atribut dataset Tesla.

Harga penutupan (Close) bergerak dari 3,16 sampai 880,02 dolar dengan rata-rata 63,67 dan standar deviasi 104,06. Rentang yang lebar ini menandakan kenaikan harga yang besar selama sepuluh tahun, dan pola seperti ini menjadi kunci pada pembahasan hasil nanti.



Gambar 12: Deret waktu harga penutupan saham Tesla 2010 sampai 2021. Harga relatif datar pada tahun-tahun awal, lalu naik tajam menjelang akhir periode.

3.2.2 Skenario A: Prediksi dari Close 7 Hari Sebelumnya

Tugas pertama meminta prediksi Close dua hari ke depan memakai Close tujuh hari sebelumnya. Karena targetnya dua langkah ke depan, `n_steps_out` diisi 2.

```
1 df_A = ts[['Close', 'Close']]
2 seq_A = df_A.astype(float).values
3 XA, yA = split_sequences(seq_A, 7, 2)
4 XA = XA.reshape((XA.shape[0], XA.shape[1] * XA.shape[2]))
5 XA_tr, XA_te, yA_tr, yA_te = train_test_split(
6     XA, yA, test_size=0.2, random_state=42, shuffle=False)
7 XA_tr = stats_features(XA_tr)
8 XA_te = stats_features(XA_te)
9
10 resA = {}
11 resA['MLP'] = mlp(XA_tr, XA_te, yA_tr, yA_te)
12 resA['KNN'] = knn(XA_tr, XA_te, yA_tr, yA_te)
13 resA['DT'] = dt(XA_tr, XA_te, yA_tr, yA_te)
14 resA['SVR'] = svm(XA_tr, XA_te, yA_tr, yA_te)
15 resA['RF'] = rf(XA_tr, XA_te, yA_tr, yA_te)
```

Listing .14: Skenario A: masukan hanya Close.

3.2.3 Skenario B: Prediksi dari Close dan Open 7 Hari Sebelumnya

Tugas kedua menambah Open sebagai fitur. Karena ada dua kolom masukan, jendela berisi 14 nilai mentah sebelum fitur statistik ditambahkan, sehingga totalnya menjadi 22 fitur.

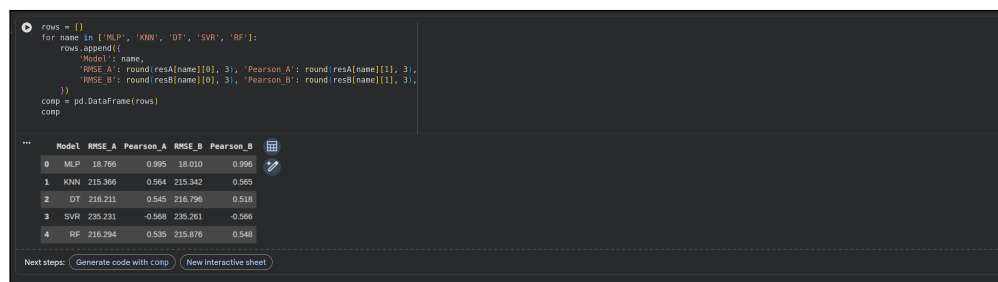
```
1 df_B = ts[['Close', 'Open', 'Close']]
2 seq_B = df_B.astype(float).values
3 XB, yB = split_sequences(seq_B, 7, 2)
4 XB = XB.reshape((XB.shape[0], XB.shape[1] * XB.shape[2]))
```

```

5 XB_tr, XB_te, yB_tr, yB_te = train_test_split(
6     XB, yB, test_size=0.2, random_state=42, shuffle=False)
7 XB_tr = stats_features(XB_tr)
8 XB_te = stats_features(XB_te)
9
10 resB = {}
11 resB['MLP'] = mlp(XB_tr, XB_te, yB_tr, yB_te)
12 resB['KNN'] = knn(XB_tr, XB_te, yB_tr, yB_te)
13 resB['DT'] = dt(XB_tr, XB_te, yB_tr, yB_te)
14 resB['SVR'] = svm(XB_tr, XB_te, yB_tr, yB_te)
15 resB['RF'] = rf(XB_tr, XB_te, yB_tr, yB_te)

```

Listing .15: Skenario B: masukan Close dan Open.



Model	RMSE_A	Pearson_A	RMSE_B	Pearson_B
0 MLP	18.766	0.995	18.010	0.996
1 KNN	215.366	0.564	215.342	0.565
2 DT	216.211	0.545	216.796	0.518
3 SVR	235.231	-0.568	235.261	-0.566
4 RF	216.294	0.535	215.876	0.548

Gambar 13: Hasil evaluasi kelima model pada kedua skenario Tesla (tabel perbandingan dari Colab).

Model	Skenario A (Close)		Skenario B (Close + Open)	
	RMSE	Pearson	RMSE	Pearson
MLP (Neural Network)	18,766	0,995	18,010	0,996
KNN	215,366	0,564	215,342	0,565
Random Forest	216,294	0,535	215,876	0,548
Decision Tree	216,211	0,545	216,796	0,518
SVR	235,231	−0, 568	235,261	−0, 566

Table .3: Perbandingan performa lima model pada dua skenario tugas Tesla (split 80:20, shuffle=False).

3.2.4 Analisis: Mengapa MLP Unggul di Sini

Selisih antara MLP dan empat model lain pada dataset Tesla terlalu besar untuk diabaikan. RMSE MLP 18,766 sementara KNN, Random Forest, dan Decision Tree semuanya di kisaran 215, sekitar sebelas kali lebih besar. Saya menelusuri penyebabnya dengan memeriksa rentang nilai data latih dan data uji.

```

1 print('y_train: min %.2f max %.2f' % (yB_tr.min(), yB_tr.max()))
2 print('y_test : min %.2f max %.2f' % (yB_te.min(), yB_te.max()))

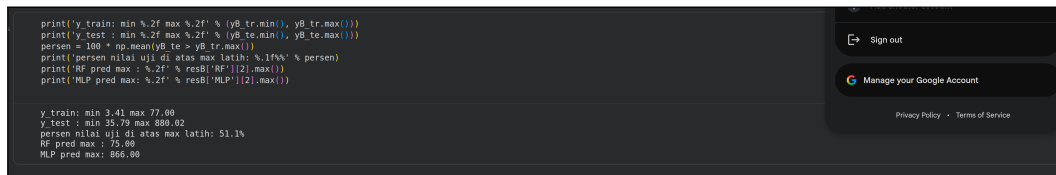
```

```

3 persen = 100 * np.mean(yB_te > yB_tr.max())
4 print('persen nilai uji di atas max latih: %.1f%%' % persen)

```

Listing .16: Memeriksa rentang target latih dan uji.



```

print('y_train: min %.2f max %.2f' % (yB_tr.min(), yB_tr.max()))
print('y_test : min %.2f max %.2f' % (yB_te.min(), yB_te.max()))
persen = 100 * np.mean(yB_te > yB_tr.max())
print('persen nilai uji di atas max latih: %.1f%%' % persen)
print('RF pred max : %.2f' % resB['RF'][:,2].max())
print('MLP pred max: %.2f' % resB['MLP'][:,2].max())

y_train: min 3.41 max 77.00
y_test : min 35.79 max 880.02
persen nilai uji di atas max latih: 51.1%
RF pred max : 75.00
MLP pred max: 866.00

```

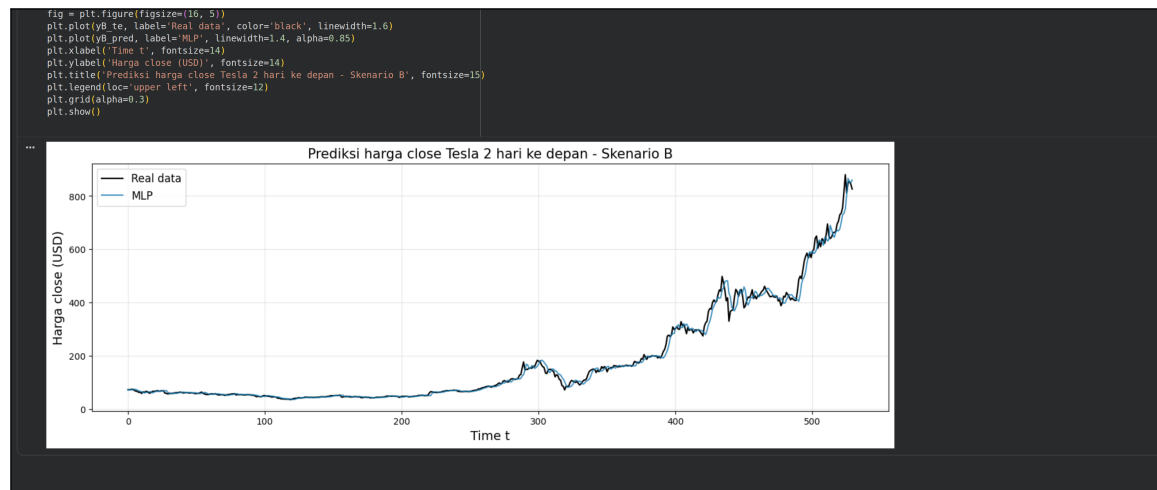
Gambar 14: Output pemeriksaan rentang data latih dan uji pada notebook Colab.

Hasilnya, target data latih hanya mencakup harga 3,41 sampai 77,00 dolar, sedangkan data uji mencakup 35,79 sampai 880,02 dolar. Sebanyak 51,1% nilai uji berada di atas harga tertinggi yang pernah dilihat model saat latihan. Ini terjadi karena pembagian datanya kronologis: 80% awal berisi periode harga murah (2010 sampai sekitar 2019), dan 20% akhir berisi lonjakan harga Tesla (2020 sampai awal 2021).

Sifat ini membedakan model. Decision Tree, Random Forest, dan KNN hanya bisa memprediksi di dalam rentang nilai yang pernah mereka lihat, karena prediksinya berasal dari rata-rata target pada data latih. Pohon tidak bisa menghasilkan nilai di luar kotak nilai yang dikenalnya. Pemeriksaan langsung menunjukkan Random Forest memprediksi maksimum 75,00 dolar, sementara MLP bisa mencapai 866,00 dolar, mendekati harga aktual 880,02. MLP bisa melakukannya karena jaringan saraf memetakan masukan lewat fungsi kontinu yang bisa menghasilkan nilai di luar rentang data latih. Karena itu di sini MLP unggul terutama karena ia satu-satunya model yang mampu mengikuti tren naik yang belum pernah dilihatnya.

SVR memberi korelasi negatif ($-0,568$). Ini terjadi karena SVR dengan parameter bawaan cenderung menghasilkan prediksi yang hampir rata, sehingga ketika harga aktual naik tajam, arah hubungannya berbalik. Tanpa normalisasi dan penyetelan parameter, SVR tidak cocok untuk data dengan rentang harga sebesar ini.

Perlu juga saya bandingkan dengan baseline naif yang memakai harga penutupan terakhir pada jendela. Baseline ini menghasilkan RMSE 17,278 dan Pearson 0,996, sedikit lebih baik daripada MLP. Artinya pada data ini, sebagian besar akurasi berasal dari sifat harga saham yang bergerak mulus dari hari ke hari, bukan dari kemampuan model menangkap pola rumit. Prediksi dua hari ke depan dengan tujuh hari riwayat praktis mirip dengan mengulang harga terakhir. Angka RMSE di bawah 20 dolar memang terlihat bagus, tetapi perlu dibaca bersama fakta bahwa rentang harga pada data uji mencapai 880 dolar.



Gambar 15: Perbandingan nilai aktual dan prediksi MLP pada Skenario B. MLP mampu mengikuti tren naik harga, sedangkan model berbasis pohon berhenti di sekitar 75 dolar.

3.2.5 Perbandingan Skenario A dan B

Penambahan Open sebagai fitur hanya mengubah hasil sedikit. Pada MLP, RMSE turun dari 18,766 menjadi 18,010 dan Pearson naik tipis dari 0,995 menjadi 0,996. Pada model lain perubahannya bahkan lebih kecil. Ini masuk akal karena harga pembukaan dan harga penutupan Tesla punya korelasi 0,9992, hampir sempurna. Harga Open hampir tidak menambah informasi baru di luar apa yang sudah dibawa Close. Jadi untuk kasus ini, menambah fitur tidak banyak menolong, dan model dengan masukan lebih sedikit sudah cukup.

Kesimpulan

Berdasarkan praktikum yang telah saya lakukan pada Pertemuan 5, saya memahami beberapa hal berikut:

1. *Forecasting* adalah proses memperkirakan nilai masa depan dari pola data masa lalu. Untuk data deret waktu, urutan data harus dijaga saat pembagian latih dan uji, sehingga `shuffle=False` dipakai agar model tidak melihat masa depan.
2. Saya berhasil mengubah deret waktu menjadi format *supervised learning* dengan teknik *sliding window*, memakai tujuh nilai sebelumnya sebagai masukan. Penambahan delapan fitur statistik (min, max, selisih, mean, std, median, kurtosis, skewness) menaikkan jumlah fitur dari 7 menjadi 15 dan memberi model gambaran distribusi tiap jendela.
3. Pada percobaan bike sharing, MLP menjadi model terbaik dengan RMSE 1190,549 dan Pearson 0,778, diikuti Random Forest (RMSE 1324,452, Pearson 0,716) dan KNN (RMSE 1357,710, Pearson 0,696). Decision Tree terlemah dari sisi Pearson (0,484) karena pohon tunggal mudah *overfitting*, dan SVR memberi RMSE tertinggi (2404,928) karena fitur tidak dinormalisasi. Baseline naif menghasilkan RMSE 1290,766, sehingga keunggulan MLP di sini nyata tetapi tidak besar.
4. Pada tugas Tesla, saya membangun dua model: prediksi Close dua hari ke depan dari Close saja, dan dari Close ditambah Open. Hasilnya MLP unggul jauh pada kedua skenario (RMSE 18,766 dan 18,010). Setelah saya telusuri, penyebabnya adalah data uji berisi periode harga yang belum pernah dilihat model saat latihan: 51,1% nilai uji berada di atas harga tertinggi data latih. Model berbasis pohon dan KNN tidak bisa memprediksi di luar rentang data latihnya, sedangkan MLP bisa mengikuti tren naik. Baseline naif (RMSE 17,278) bahkan sedikit lebih baik dari MLP, tanda bahwa akurasi di sini banyak ditopang oleh sifat harga saham yang bergerak mulus.
5. Saya memahami bahwa RMSE dan Pearson harus dibaca bersama. RMSE menunjukkan besar kesalahan, sedangkan Pearson menunjukkan seberapa mirip pola prediksi dengan pola aktual. Nilai RMSE yang rendah belum tentu berarti model menangkap pola yang sulit, seperti terlihat pada kasus Tesla.

6. Saya juga belajar bahwa menambah fitur tidak selalu menolong. Menambahkan Open hanya memperbaiki hasil sedikit karena korelasinya dengan Close hampir sempurna (0,9992), sehingga informasinya sebagian besar sudah terwakili.

Daftar Pustaka

- [1] E. Retnoningsih and A. Pramudita, “Mengenal Machine Learning Dengan Teknik Supervised dan Unsupervised Learning Menggunakan Python,” *BINA INSANI ICT Journal*, pp. 156–165, 2020.
- [2] A. Santoso, H. Abijono, and D. Anggreini, “Algoritma Supervised Learning dan Unsupervised Learning dalam Pengolahan Data,” *Jurnal Teknologi Terapan*, pp. 315–318, 2021.
- [3] T. Vimala and S. Nugroho, “Forecasting Penjualan Obat Menggunakan Metode Single, Double, dan Triple Exponential Smoothing (Studi Kasus: Apotek Mandiri Medika),” *Jurnal Penerapan Teknologi Informasi dan Komunikasi*, pp. 90–99, 2022.
- [4] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Tersedia: <https://otexts.com/fpp3/>.
- [5] T. Chai and R. R. Draxler, “Root mean square error (RMSE) or mean absolute error (MAE)? Arguments against avoiding RMSE in the literature,” *Geoscientific Model Development*, vol. 7, no. 3, pp. 1247–1250, 2014. [Online]. Tersedia: <https://doi.org/10.5194/gmd-7-1247-2014>.
- [6] J. Benesty, J. Chen, Y. Huang, and I. Cohen, “Pearson Correlation Coefficient,” in *Noise Reduction in Speech Processing*, Berlin, Heidelberg: Springer, 2009, pp. 1–4. [Online]. Tersedia: https://doi.org/10.1007/978-3-642-00296-0_5.
- [7] Scikit-learn Developers, “sklearn.neural_network.MLPRegressor.” [Online]. Tersedia: https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPRegressor.html.
- [8] Scikit-learn Developers, “sklearn.neighbors.KNeighborsRegressor.” [Online]. Tersedia: <https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsRegressor.html>.
- [9] Scikit-learn Developers, “sklearn.tree.DecisionTreeRegressor.” [Online]. Tersedia: <https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeRegressor.html>.

- [10] Scikit-learn Developers, “sklearn.svm.SVR.” [Online]. Tersedia: <https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVR.html>.
- [11] Scikit-learn Developers, “sklearn.ensemble.RandomForestRegressor.” [Online]. Tersedia: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html>.
- [12] Scikit-learn Developers, “Metrics and scoring: quantifying the quality of predictions.” [Online]. Tersedia: https://scikit-learn.org/stable/modules/model_evaluation.html.
- [13] Scikit-learn Developers, “Preprocessing data.” [Online]. Tersedia: <https://scikit-learn.org/stable/modules/preprocessing.html>.
- [14] Scikit-learn Developers, “sklearn.model_selection.train_test_split.” [Online]. Tersedia: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html.
- [15] The pandas Development Team, “pandas.read_csv.” [Online]. Tersedia: https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html.
- [16] NumPy Developers, “numpy.array.” [Online]. Tersedia: <https://numpy.org/doc/stable/reference/generated/numpy.array.html>.
- [17] SciPy Developers, “scipy.stats.pearsonr.” [Online]. Tersedia: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.pearsonr.html>.
- [18] SciPy Developers, “scipy.stats.kurtosis.” [Online]. Tersedia: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.kurtosis.html>.
- [19] SciPy Developers, “scipy.stats.skew.” [Online]. Tersedia: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.skew.html>.
- [20] Matplotlib Development Team, “Matplotlib Documentation.” [Online]. Tersedia: <https://matplotlib.org/stable/index.html>.
- [21] M. Waskom, “seaborn: statistical data visualization.” [Online]. Tersedia: <https://seaborn.pydata.org/>.
- [22] Python Software Foundation, “Python 3 Documentation.” [Online]. Tersedia: <https://www.python.org/doc/>.
- [23] Wikipedia, “Multilayer perceptron.” [Online]. Tersedia: https://en.wikipedia.org/wiki/Multilayer_perceptron.
- [24] Wikipedia, “k-nearest neighbors algorithm.” [Online]. Tersedia: https://en.wikipedia.org/wiki/K-nearest_neighbors_algorithm.

- [25] Wikipedia, “Decision tree learning.” [Online]. Tersedia: https://en.wikipedia.org/wiki/Decision_tree_learning.
- [26] Wikipedia, “Support vector machine.” [Online]. Tersedia: https://en.wikipedia.org/wiki/Support_vector_machine.
- [27] Wikipedia, “Random forest.” [Online]. Tersedia: https://en.wikipedia.org/wiki/Random_forest.
- [28] Wikipedia, “Sliding window.” [Online]. Tersedia: https://en.wikipedia.org/wiki/Sliding_window.
- [29] Wikipedia, “Feature engineering.” [Online]. Tersedia: https://en.wikipedia.org/wiki/Feature_engineering.
- [30] L. N. Pathi, “Bike Sharing Dataset.” Kaggle. [Online]. Tersedia: <https://www.kaggle.com/datasets/lakshmi25npathi/bikesharing-dataset>.
- [31] Timoboz, “Tesla Stock Data from 2010 to 2020.” Kaggle. [Online]. Tersedia: <https://www.kaggle.com/datasets/timoboz/tesla-stock-data-from-2010-to-2020>.
- [32] G. Alfarizi, “data_science_practice (bikesharing_day.csv).” GitHub. [Online]. Tersedia: https://github.com/ganjar87/data_science_practice.
- [33] P. Jain, “Stock-Prediction-using-different-models (TSLA.CSV).” GitHub. [Online]. Tersedia: <https://github.com/0xpranjal/Stock-Prediction-using-different-models>.